

Project Vitriol – NEA Analysis

Problem Identification

Currently, the development of tile-based 2D game worlds faces a number of structural and architectural limitations which, while not always immediately visible, disproportionately affect beginner and intermediate creators. Although there are many map editors and game engines available, there is a fundamental mismatch between ease of use, architectural cleanliness, and runtime integration. The tools with the most flexibility often require significant technical understanding of engine internals, rendering pipelines, or file formats. Conversely, tools that are simple to use frequently lack scalability, performance stability, or safe persistence mechanisms.

This creates a development environment in which creative intent is often constrained not by imagination, but by the limitations of tooling design. In many cases, the learning curve is not steep because of conceptual difficulty in map design itself, but because of fragmented workflows, inefficient rendering models, or fragile data handling systems. As a result, technical familiarity becomes prioritised over creative flow, discouraging users who lack deep engine knowledge from progressing beyond small-scale experimentation.

Fundamental Limitations

- **Dependence on External Integration** – Many widely used tile map editors operate as isolated tools rather than complete systems. They allow the user to construct a visual map, but require manual integration into a separate runtime engine for playtesting. This separation forces creators to understand file exports, coordinate systems, collision implementation, and rendering logic that are not inherently part of map design. Rather than the architecture serving the creator, the creator is required to adapt to the architecture. This introduces unnecessary complexity, particularly for beginners who simply wish to build and test a playable environment.
- **The Feel of Disjointed Workflow** – Managing map data often involves switching between multiple disconnected conceptual systems: visual tile editing, collision configuration, event logic, and runtime testing. In many environments, these systems are not tightly coupled, meaning that what is visually constructed does not always reflect what is functionally playable. For example, a tile may appear walkable in the editor but behave differently in the runtime due to separate collision handling. This fragmentation increases cognitive overhead, as the user must mentally synchronise multiple layers of logic. Over time, this discourages experimentation, because small changes require disproportionate effort to verify.

- **Performance Fragility at Scale** – A common architectural weakness in lightweight editors is naive rendering strategy. Rendering entire maps every frame, rather than calculating only the visible viewport region, leads to unnecessary computation. While this may be acceptable for small maps, performance degrades significantly as dimensions increase. Within UI-driven frameworks such as WPF, this problem becomes more pronounced due to the overhead of managing large numbers of visual elements. Without deliberate optimisation strategies such as viewport-only rendering or region recalculation, scalability becomes constrained and the editing experience deteriorates.
- **Persistence and Data Fragility** – Many beginner-oriented tools rely on loosely structured JSON or text-based storage formats without strict schema validation, version tracking, or defensive loading behaviour. This creates vulnerabilities when:
 - Files are moved between devices
 - Asset paths are changed
 - Map dimensions are modified
 - Format updates occur

In the absence of controlled autosave policies and validation layers, users risk overwriting valid data with incomplete states. The problem is not the use of JSON itself, but the lack of architectural discipline surrounding its structure and lifecycle.

- **Limited Structured Guidance** – Existing systems often expose their full feature set immediately, without distinguishing between foundational and advanced concepts. Beginners are presented with layers, file structures, rendering concerns, and collision systems simultaneously. Without a progressive workflow model, users are forced to learn through error, which shifts focus away from design and toward troubleshooting. Over time, this erodes confidence and reduces creative momentum.

Supporting Evidence:

Across community discussions surrounding tile-based editors and small engine toolchains, recurring frustrations include runtime mismatch between editor and gameplay, map lag at larger scales, broken asset references when projects are transferred, and inconsistent undo behaviour across editing modes. These issues frequently appear in help threads and development blogs, indicating that the core limitation is not feature absence, but architectural misalignment.

The pattern that emerges is clear: when tools prioritise rapid feature expansion over structural clarity and integration, creators inherit instability and fragmentation. This

demonstrates that the barrier to creativity is rarely a lack of imagination, but rather friction introduced by the design of the development environment itself.

Client and User Description

The main client for this project is the community of beginner-to-intermediate 2D game developers who wish to construct playable tile-based environments without needing to architect a complete engine pipeline themselves. These users typically lack deep familiarity with rendering systems, file format design, or runtime architecture, yet still expect professional-level stability and responsiveness from their tools.

The client's requirements stem from a prioritisation of accessibility and creative flow over low-level technical complexity. Rather than focusing on engine internals, these creators aim to build environments visually, test them immediately, and iterate rapidly without fear of instability or data corruption. Therefore, the system must balance architectural discipline internally while presenting a controlled and intuitive interface externally.

Key Client Objectives:

- **Elimination of Main Barriers** – The primary objective is to remove the friction that exists between map design and runtime playtesting. Users should not need to manually integrate exported files into a separate engine, configure collision logic through code, or interpret file schemas to see their work function. The editor and runtime must share a unified data model so that a created map is immediately playable. By abstracting file handling, rendering calculations, and collision indexing away from the user, the system enables immediate engagement with creative tasks rather than technical configuration.
- **Intuitive Workflow Design** – The interface should reflect the natural order in which maps are built: tile layout first, collision refinement second, event placement third. Tools must be grouped by workflow relevance rather than technical categorisation. Editing modes (tile, collision, event) should be clearly separated to prevent conceptual overlap, while still maintaining visual cohesion within a single viewport. Immediate feedback is critical; painting a tile must update the screen instantly, collision overlays must clearly indicate blocked regions, and events must visibly occupy positions on the grid upon placement. The system's terminology and layout should align with how creators think about worlds, not how the software internally stores them.
- **Confidence Building Through Structural Safety** – Users must feel safe experimenting. This requires a robust undo/redo system that applies consistently across tile edits, collision changes, and event placement. Autosave behaviour must be carefully controlled: newly created maps require an initial manual save before autosave activation, preventing accidental file creation, while still protecting ongoing work from unexpected shutdown. Validation during

file loading must prevent crashes caused by malformed JSON or missing tileset paths. Error messages should not simply report failure, but clarify cause and suggest resolution.

- **Progressive Mastery** – The system should allow beginners to engage with foundational features without being overwhelmed by advanced concerns. A user should be able to paint tiles without needing to understand collision systems, and test maps in the runtime without understanding rendering loops. As familiarity increases, users can begin exploring event systems, camera behaviour, and larger map scaling. This layered exposure reduces cognitive overload and encourages sustained development rather than abandonment due to complexity.

End User Profiles

Beginner Creator:

No formal experience with engine architecture. Requires clear editing modes, error prevention, predictable saving behaviour, and immediate visual confirmation of actions.

Intermediate Designer:

Some familiarity with development tools. Requires performance scalability for larger maps, deterministic runtime behaviour, efficient editing workflows (shortcuts and drag-based tools), and portable file structures.

User Needs Analysis

Beginner Creator Needs (Primary Users):

- **Need Guided Structural Flow:**
The user should be able to follow a clear progression:
Create Map -> Paint Tiles -> Define Collision -> Place Events -> Save -> Test in Runtime.
The system must not require understanding of rendering algorithms or file hierarchies to complete this cycle.
- **Need for Error Prevention:**
The software must validate map dimensions before creation, verify tileset references before rendering, and handle corrupted files gracefully. Preventative validation reduces frustration and preserves confidence.
- **Need for Immediate Feedback:**
Every action must be visible instantly within the viewport. Collision overlays should toggle clearly, and event markers should display at exact grid positions, reinforcing alignment between data and visual state.
- **Need for Safe Experimentation:**
Undo/redo must be deterministic and reversible. Autosave must protect

progress without creating file clutter or overwriting unsaved maps. The system must ensure that no editing action irreversibly damages the map structure.

- **Need for Progressive Complexity:**

The interface should allow users to focus on one conceptual system at a time (tiles, collision, events), preventing cognitive overload and reducing accidental cross-editing.

Intermediate Creator Needs (Secondary Users):

- **Need for Efficient Workflow:**

Keyboard shortcuts (e.g., undo/redo), drag-to-paint functionality, smooth zoom and pan controls, and responsive viewport rendering to reduce friction during large-scale edits.

- **Need for Creative Flexibility Within Structure:**

While the system abstracts engine internals, it must not overly restrict map structure. Event data should support extension through command lists, and map formats should remain adaptable via versioned JSON schemas.

- **Need for Data Integrity:**

Maps must include format version fields and consistent schema definitions. Relative tileset paths ensure portability between machines without broken references.

- **Need for Performance Stability:**

Maps exceeding typical beginner scale must remain responsive. Viewport-only rendering must prevent full-map redraw overhead, ensuring scalability as complexity increases.

- **Need for Runtime Consistency:**

Collision detection and camera behaviour must match editor expectations precisely. A deterministic runtime loop ensures that gameplay behaviour reflects map configuration without unexpected divergence.

Usability Requirements

The System Usability Scale (SUS) will be used to evaluate interface effectiveness, with a target score above 80% across user segments.

Core usability expectations include:

- Users must be able to create and test a playable map within a single session without external documentation.
- Editing modes must be visually distinguishable and logically separated.
- Undo/redo must function consistently across all editing actions.
- Save behaviour must clearly differentiate between first manual save and subsequent autosave operations.

- File portability between systems must not break tileset references or corrupt map data.

User feedback should indicate increased confidence over time, a clear understanding of editing modes, and the ability to recover from mistakes without external assistance.

Research Into Existing Systems

RPG Maker (XP / MV / MZ)

- **Architecture Analysis** – RPG Maker provides an integrated editor and runtime system, which superficially appears aligned with beginner-friendly development. However, its architecture is proprietary and tightly coupled, meaning map data, event systems, collision handling, and rendering behaviour are deeply embedded within the engine's internal structure. The developer does not directly control how tiles are batched, how collision grids are evaluated, or how map data is internally cached. This creates architectural opacity: while early development is simplified, extensibility becomes constrained. Modifications beyond predefined behaviours require scripting within the engine's embedded language (Ruby or JavaScript depending on version), introducing a dependency on internal APIs that were not designed for modular extension. Over time, this leads to version migration issues, where changes in engine internals affect previously stable projects.
- **Data and Persistence Model** – Map files in RPG Maker are stored in engine-specific formats that abstract away structural details. While convenient, this reduces transparency. Developers cannot easily validate schema, enforce version compatibility, or implement custom migration strategies. Event command systems are stored in structured but engine-bound representations, limiting portability outside the ecosystem. Additionally, data storage is distributed across map files, database files, and scripting layers, creating implicit dependencies between components.
- **User Experience Assessment** – Although marketed as accessible, the interface separates map editing, event configuration, database management, and playtesting into distinct conceptual modes. The event editor in particular introduces a non-spatial abstraction layer: event logic is constructed in command lists separate from the visual tile grid, requiring users to mentally associate spatial placement with conditional logic. This separation increases cognitive load. Furthermore, menu structures are often grouped by engine subsystem rather than creator workflow, meaning users must navigate technical categorisation rather than creative sequencing.
- **Technical Barriers** – Advanced customisation requires scripting knowledge and familiarity with engine execution order. Simple alterations to collision rules, rendering behaviour, or camera systems cannot be achieved purely through the

visual interface. Error reporting, when scripting is involved, exposes raw engine traces rather than user-oriented guidance. This creates a learning divide between users who remain within predefined functionality and those who attempt deeper modification.

- **Performance Considerations** – Rendering optimisation is handled internally and abstracted from the user. While suitable for typical map sizes, users cannot explicitly enforce viewport-only rendering strategies or optimise draw regions manually. As a result, scalability is bounded by engine design decisions rather than deliberate architectural planning.

Tiled Map Editor

- **Architecture Analysis** – Tiled is an open-source, standalone tile map editor that supports multiple layers (tile layers, object layers, image layers) and exports to structured formats such as JSON and TMX. Architecturally, it is modular and extensible within its editing domain. However, it deliberately excludes runtime functionality. This design philosophy positions Tiled as a content creation tool rather than an integrated system.
- **Data Model Strength and Limitation** – Tiled's exported JSON is structured and relatively clean, but it is editor-centric rather than runtime-centric. Collision data is typically represented through object layers or tile properties, requiring interpretation by the consuming engine. While flexible, this shifts architectural responsibility onto the developer. There is no enforced schema discipline beyond what the editor produces; developers must implement their own validation, versioning, and migration strategies.
- **Workflow Fragmentation** – Because Tiled does not include a runtime, users must integrate exported maps into a separate engine manually. This introduces multiple friction points:
 - Coordinate translation
 - Tile index mapping
 - Collision grid reconstruction
 - Camera and viewport logic implementation

The visual design environment and the runtime execution environment remain separate, requiring iterative adjustment across two independent systems.

- **Technical Barriers** – Although map editing itself is straightforward, meaningful use in a playable project demands knowledge of rendering loops, JSON parsing, collision algorithms, and asset management. The barrier shifts from map design to integration engineering.
- **Performance Responsibility** – Tiled does not prescribe rendering optimisation strategies. If a developer implements naive full-map redraws in their runtime, performance degrades. Thus, scalability depends entirely on the architectural competence of the integration layer rather than the editor.

Unity (2D Tilemap Workflow)

- **Architecture Analysis** – Unity is a general-purpose game engine built around a component-based architecture. While it supports 2D tilemaps, these exist within a broader ecosystem designed primarily for 3D rendering. Tilemap components are attached to GameObjects, which exist within scene hierarchies governed by lifecycle callbacks (Update, FixedUpdate, etc.). This abstraction model introduces complexity disproportionate to the needs of lightweight 2D map editing.
- **System Overhead** – Unity’s architecture includes physics engines, lighting systems, audio subsystems, asset import pipelines, and serialisation frameworks that operate regardless of whether a project requires them. For simple tile-based map workflows, this introduces unnecessary computational and conceptual overhead. Memory usage and build sizes are typically significantly larger than specialised 2D solutions.
- **User Experience Assessment** – The editor interface distributes responsibilities across Scene view, Game view, Hierarchy, Inspector, and Project windows. While powerful, this layout demands understanding of multiple interacting subsystems. Tilemaps must be combined with collider components, camera scripts, and input systems manually. There is no enforced workflow ordering; beginners must determine architectural structure independently.
- **Technical Barriers** – Implementing deterministic collision systems, camera deadzone behaviour, event triggering, and viewport optimisation requires C# scripting. Knowledge of draw calls, batching, and garbage collection becomes relevant when scaling maps. This exceeds the intended scope for beginner creators seeking simple map-to-runtime iteration.
- **Versioning and Serialisation Risks** – Unity’s prefab and scene serialisation systems can change between versions, occasionally causing migration issues. Large projects may encounter conflicts between tilemap colliders and physics layers, requiring architectural debugging rather than design focus.

Lightweight Standalone Map Editors

- **Tool Analysis** – Many lightweight editors prioritise rapid tile placement with minimal interface complexity. However, these tools often lack strict persistence models. Map files may not include version identifiers, schema validation, or enforced relative asset referencing. As a result, portability between systems becomes fragile.
- **Undo/Redo Limitations** – In some tools, undo functionality applies only to tile painting and not to object placement or property editing. This inconsistency undermines user confidence and discourages experimentation.

- **Integration and Schema Drift** – Without a shared runtime model, schema evolution can break older maps. Changes to tile indexing or layer definitions may invalidate existing files. Asset paths may be stored absolutely rather than relatively, causing reference failures when directories change.
- **Architectural Simplicity vs Structural Discipline** – While accessible, these editors frequently prioritise simplicity over robustness. They lack defensive loading, controlled autosave behaviour, and explicit version management — all of which are critical for long-term project stability.

Market Gap:

At present, no single lightweight solution simultaneously provides:

- Beginner-accessible visual editing
- Integrated runtime playtesting within the same architectural model
- Explicit viewport-only rendering in both editor and runtime
- Strict JSON schema discipline with version tracking
- Relative asset referencing for portability
- Command-pattern undo/redos consistency across all editing modes
- Deterministic runtime loop aligned with editor expectations

Existing systems either prioritise accessibility at the expense of architectural clarity, or prioritise extensibility at the expense of usability.

This reveals a clear gap for a system that balances disciplined software engineering principles with tightly integrated, creator-oriented workflow design.

Problem Statement

Currently, the ecosystem surrounding tile-based 2D map creation suffers from a fundamental mismatch between creator needs and tool architecture. While there are numerous editors and engines available, very few are designed with an intentional balance between accessibility, structural discipline, and runtime integration. This mismatch creates a barrier that disproportionately affects beginner and intermediate developers – ironically, the very group most motivated to experiment with world-building and interactive design.

The core issue is not the absence of features. In fact, many existing tools are extremely feature-rich. The problem lies in how those features are architected and exposed. Many systems prioritise extensibility, abstraction, and technical flexibility over cohesive workflow and structural clarity. As a result, creators are often forced to adapt to the tool's internal logic rather than the tool adapting to their creative process.

Because of this imbalance, the following systemic issues emerge:

- **Experimentation is Discouraged Through Architectural Fragility** – In many workflows, errors manifest as broken file references, corrupted exports, inconsistent runtime behaviour, or unexplained rendering anomalies. When undo systems are incomplete or autosave behaviour is unpredictable, users develop a fear of making structural changes. The risk of accidentally breaking a map increases as complexity grows. This discourages experimentation, which is foundational to creative development. Instead of encouraging iteration, the tool becomes something to cautiously manage.
- **Fragmented Workflow Reinforces Cognitive Overhead** – Existing solutions frequently separate editing and runtime environments, or isolate logical systems from visual ones. Collision handling may not directly correspond to visual tile placement. Event systems may exist as abstract object lists separate from spatial context. This fragmentation forces users to maintain a mental model of how separate subsystems interact. As complexity increases, so does the cognitive burden required to maintain consistency between these disconnected representations.
- **Integration Responsibility is Shifted Onto the User** – When editors do not share a unified data model with their runtime environments, users must manually bridge the gap. This may involve interpreting JSON exports, rebuilding collision grids, implementing camera logic, or translating coordinate systems. Instead of the tool guaranteeing structural alignment, integration becomes a manual engineering task. This effectively raises the barrier to entry for users who lack formal engine development experience.
- **Scalability is Often an Afterthought** – Many lightweight editors function adequately at small scales but degrade as map size increases. Without viewport-based rendering or efficient region recalculation, performance suffers due to unnecessary full-map redraws. When performance issues emerge, users must debug rendering inefficiencies that stem from architectural design choices they never consciously made.
- **Learning is Reactionary Rather Than Structured** – Documentation for many systems focuses on solving isolated issues rather than explaining architectural reasoning. Users learn how to fix specific symptoms rather than understanding the underlying model. As a result, their knowledge becomes context-dependent and fragile. When faced with scenarios outside documentation examples, confidence collapses because the conceptual foundation was never established.

These issues demonstrate that the limitation is not creative capacity, nor feature availability, but rather the structural misalignment between user workflow and tool architecture.

Therefore, a new solution must be deliberately designed around creator-centric principles rather than engine-centric abstraction. Such a solution must provide:

- **Progressive Complexity** – The interface must allow users to engage with foundational tasks (tile painting and saving) before encountering more advanced systems (collision indexing, event logic, runtime behaviour). Advanced configuration should exist but not overwhelm initial workflow.
- **Integrated Data Management** – All map elements (tile data, collision data, event data) must exist within a unified schema shared between editor and runtime. Changes made in the editor must be directly consumable by the runtime without translation layers or manual interpretation.
- **Runtime-Editor Cohesion** – The runtime should reflect exactly what is constructed in the editor, eliminating discrepancies between visual representation and gameplay behaviour.
- **Error-Resistant Design** – The system must implement controlled autosave policies, command-pattern undo/redo, defensive JSON loading, and clear error messaging. Destructive actions should require confirmation, and persistence mechanisms must prioritise structural safety.
- **Scalable Rendering Architecture** – Both editor and runtime must implement viewport-only rendering strategies to ensure consistent performance as map dimensions increase.
- **Sustainable Modular Architecture** – The internal codebase must separate concerns between rendering, input handling, data persistence, and editing logic. This modular structure ensures that extending functionality does not destabilise existing behaviour.

In summary, the required solution is not simply “another map editor,” but a deliberately architected system that integrates disciplined software engineering with accessible creative workflow. Only by aligning structural integrity with usability can the barrier between creative intention and technical execution be meaningfully reduced.

Intended Solution

The intended solution provides a tightly integrated tile-based map editor and runtime environment built around architectural clarity, performance scalability, and creator-centric workflow design. Rather than overwhelming users with excessive abstraction or feature density, the system establishes sensible structural defaults while preserving extensibility through disciplined design.

The core philosophy is to separate internal architectural complexity from external user interaction. Rendering logic, collision indexing, persistence formatting, and runtime execution are handled internally through modular components, ensuring that technical

mechanisms do not interrupt creative flow. At the same time, these systems are deliberately engineered to remain transparent, predictable, and scalable.

Core System Architecture

Editor Application

The editor is implemented using WPF, leveraging a structured MVVM-inspired separation between interface components and logic handling. The interface is centred around a single unified viewport, ensuring that all editing operations remain spatially contextualised.

The system is built around progressive workflow exposure rather than feature overload.

- **Mode-Based Editing Model** – The editor separates interaction into clearly defined modes: Tile Editing, Collision Editing, and Event Editing. This prevents cross-system interference and reduces cognitive overload. Each mode modifies a distinct layer within the map's data model while sharing a common rendering viewport.
- **Layered Map Structure** – The map is composed of independent layers:
 - Tile layer (visual data)
 - Collision layer (walkability grid)
 - Event layer (position-based triggers)

This separation ensures that visual representation does not become tightly coupled to logical behaviour, preventing accidental structural corruption during editing.

- **Unified Viewport Rendering** – All editing operations occur within a single rendering surface. The viewport dynamically calculates visible tile regions and renders only those within the camera bounds. This design preserves performance even as map dimensions increase.
- **Command-Based Editing System** – All editing actions are encapsulated as command objects supporting execution and reversal. This enables deterministic undo/redo functionality across tile painting, collision toggling, and event placement. The command model ensures that state transitions are controlled and reversible.
- **Controlled Autosave Policy** – The system differentiates between:
 - Newly created maps (manual save required before autosave activation)
 - Existing maps (autosave enabled after initial save)
 - Sample maps (continuous autosave permitted)

This prevents file clutter and accidental overwriting while still protecting against data loss

- **Real-Time Validation** – Map dimensions, tile indices, and file paths are validated before persistence. Defensive loading prevents crashes caused by malformed or outdated map files.

Runtime Application

The runtime is deliberately lightweight and purpose-built for deterministic playback of maps created in the editor. It shares the same data model as the editor, eliminating format translation and integration overhead.

- **Deterministic Update Loop** – A fixed timestep game loop separates update logic from rendering frequency. This ensures consistent behaviour regardless of hardware variation.
- **Viewport-Only Rendering** – The runtime calculates visible tile bounds based on camera position and renders only necessary regions. This avoids full-map redraw inefficiencies and maintains stable frame rates even on larger maps.
- **Grid-Based Collision System** – Collision detection operates on a discrete grid aligned with the collision layer defined in the editor. Movement resolution ensures predictable traversal without clipping or inconsistent edge cases.
- **Camera Follow System** – The runtime includes a camera abstraction that defines the visible world region and smoothly follows the player within map bounds. This maintains visual stability while preserving spatial context.
- **Layered Rendering Order** – Rendering order is strictly defined to ensure correct visual hierarchy between background tiles, event markers, and player sprites.

Data Management Strategy

The system uses structured JSON files as its persistence format. Rather than relying on loosely defined schemas, the design enforces structural discipline.

- **Unified Data Model** – Both editor and runtime consume the same DTO structures, eliminating schema drift and translation inconsistencies.
- **Format Version Field** – Each map file includes a version identifier, allowing for controlled evolution of file structure without invalidating older maps.
- **Relative Asset Referencing** – Tileset paths are stored relative to project directories, ensuring portability between devices without broken references.
- **Layer Isolation Within JSON** – Tile, collision, and event data are stored in separate arrays within the same file, maintaining structural clarity and preventing cross-contamination of concerns.

Performance and Quality Targets

The solution is engineered to maintain consistent responsiveness on mid-range hardware.

- **Rendering Optimisation** – Viewport-bound tile rendering reduces unnecessary draw operations. This ensures editing and runtime playback remain responsive even for maps exceeding 100×100 tiles.

- **Memory Management Discipline** – Image assets are loaded once and reused rather than recreated per frame. Disposal routines ensure that unused resources do not accumulate.
- **Startup Efficiency** – Map loading is performed in structured stages, minimising blocking operations and ensuring startup times remain within acceptable limits for typical project sizes.
- **Frame Stability Target** – Runtime performance aims to maintain approximately 60 FPS on standard integrated graphics configurations.

Architectural Quality Attributes

The system is explicitly designed around the following quality principles:

- **Learnability** – Clear editing modes, visible overlays, and direct map-to-runtime integration reduce the time required to achieve a playable result.
- **Efficiency** – Drag-to-paint tools, keyboard shortcuts for undo/redo, and responsive viewport navigation reduce repetitive friction.
- **Error Tolerance** – Command-based undo/redo, defensive JSON loading, and controlled autosave policies protect against irreversible mistakes.
- **Consistency** – Shared data models between editor and runtime ensure predictable behaviour. Rendering order, collision evaluation, and camera logic behave deterministically.
- **Scalability** – Viewport-only rendering and structured layer separation prevent performance degradation as map complexity increases.

Functional Requirements

Technical Constraints

Hardware Compatibility Requirements

- **Target Specification** – The project must perform reliably on systems equipped with Intel UHD 620 integrated graphics, 8GB RAM, and mechanical hard drives. This ensures accessibility for users without dedicated GPUs or SSD storage.
- **Rendering Optimisation Strategy** – The system must implement viewport-only tile rendering in both editor and runtime applications. Only tiles within the visible camera region may be drawn per frame. This minimises unnecessary redraw operations and preserves frame stability as map dimensions increase.
- **Memory Management Discipline** – Image assets (tilesets) must be loaded once and reused rather than recreated per frame. Resource disposal must occur when maps are closed or replaced, preventing memory accumulation over extended editing sessions.

Software Dependency Management

- **.NET Framework Constraints** – The system must maintain compatibility with .NET 8+, while avoiding reliance on unstable preview features. This ensures long-term maintainability and stable runtime behaviour.
- **Third-Party Libraries** – External dependencies must be minimal and selected based on performance stability and compatibility with WPF. The architecture should avoid unnecessary abstraction layers that complicate debugging or migration.
- **Platform Isolation** – The system must remain Windows-compatible (Windows 11 target environment) without requiring platform-specific native integrations that complicate deployment.

Challenges Involving Resource Allocation

- **Solo Development Effort** – As a single-developer project, architectural modularity must be prioritised to reduce refactoring overhead. Shared DTO models between editor and runtime minimise duplication.
- **Structured Knowledge Management** – Architectural decisions must be documented to ensure clarity of rendering logic, persistence structure, and collision behaviour. This supports maintainability and extension.

ID	Capability	Priority
FR1	Create new maps with defined dimensions and initialise layered structure	High
FR2	Visually paint, erase, and modify tile data within a grid-based system	High
FR3	Collision Editing – Define walkable and blocked tiles via a dedicated collision layer	High
FR4	Event Placement – Add, edit, and remove position-based event triggers on the map	High
FR5	Map Persistence – Save and load maps using structured JSON with format versioning	Medium
FR6	Runtime Playback – Load maps into runtime and render with camera-based viewport logic	Medium
FR7	Player Movement & Collision – Handle deterministic grid-based collision detection	High
FR8	Editing History – Implement command-based Undo/Redo across tile, collision, and events	High
FR9	Autosave Control – Activate controlled autosave after initial manual save	Medium

FR10	Performance Stability – Maintain approximately 60 FPS on target specification hardware	High
FR11	Relative Asset Referencing – Store tileset paths relative to project directory	Medium
FR12	Layer Rendering Order – Enforce correct drawing hierarchy in editor and runtime	Medium

Non-Functional Requirements

Quality Attribute	Requirement	Measurable Target
Performance	Frame Rate Stability Map Loading Efficiency	60 FPS $\pm$ 5% variance during runtime on target hardware 2s- for 100x100 tile maps
Usability	Interface Intuitiveness Task Completion Rate	SUS Score 80%+ 85%+ successful completion of core workflow tasks
Maintainability	Code Structure Clarity Modular Architecture	Clear separation between Editor, Runtime, and Shared Models No circular dependencies between major components
Reliability	Application Stability Data Integrity	99%+ crash-free sessions during structured testing No data corruption during save/load cycles
Compatibility	Operating System Support	Full windows 11 compatibility (.NET 8+)
Scalability	Map Dimension Handling Rendering Efficiency	Stable performance for maps 100x100+ tiles Viewport-only rendering prevents full-map redraw overhead

Data Requirements and Modelling

Data Strategy

The system employs a structured, file-based data architecture designed to support both immediate usability and long-term project stability. All map data is stored using a clearly defined JSON schema shared between the editor and runtime applications. This

unified model ensures consistency across editing and execution environments while maintaining portability and transparency.

The architecture prioritises:

- Clear separation between visual and logical data
- Structural validation during load and save operations
- Version-controlled schema evolution
- Direct interoperability between editor and runtime

By maintaining a disciplined but lightweight data structure, the system supports scalable map development without introducing unnecessary architectural overhead.

Domain Model Design

The central data structure is a Map DTO (Data Transfer Object), which directly represents the conceptual components of a tile-based world.

Each map file contains:

- Map Metadata (name, dimensions, format version)
- Tilesset Reference (stored as a relative file path)
- Tile Layer Data (grid of tile indices)
- Collision Layer Data (grid defining walkability)
- Event Layer Data (collection of position-based event objects)

This structure mirrors the mental model of map construction, reducing the distance between user interaction and internal representation.

Layered Map Representation

- **Tile Layer**
Stores integer indices referencing positions within the tileset. This layer is purely visual and does not encode gameplay logic.
- **Collision Layer**
Stored as a grid aligned with tile coordinates, defining whether each tile is traversable. This separation ensures gameplay logic remains independent of visual appearance.
- **Event Layer**
Stored as a list of objects containing grid position and associated properties. Events are spatially anchored but logically independent from tile and collision data.

By isolating these concerns, the system prevents unintended cross-modification and preserves structural clarity.

Data Integrity Framework

Structural Validation

Validation is enforced at key lifecycle points:

- Tile and collision arrays must match declared map dimensions.
- Event positions must fall within valid bounds.
- Tile indices must correspond to valid tileset coordinates.
- Tileset paths must resolve correctly before rendering.

Invalid states are intercepted before runtime execution, preventing crashes and undefined behaviour.

Schema Versioning

Each map file includes a format version field. This enables controlled structural evolution while preserving compatibility with earlier map versions.

The schema is designed to be extensible, allowing additional properties to be appended without restructuring the entire model.

Performance-Oriented Data Handling

The data model directly supports rendering optimisation strategies:

- Grid-based storage enables efficient calculation of visible tile ranges.
- Layer separation allows selective rendering without recomputation of unrelated data.
- Tileset images are loaded once per session and reused, preventing repeated allocation.

This alignment between storage model and rendering logic ensures consistent performance as map dimensions increase.

Storage Efficiency Measures

- **Compact JSON Structure** – Tile and collision grids are stored in streamlined array formats, minimising file size without sacrificing readability.
- **Deterministic Serialisation** – Saving the same in-memory structure consistently produces predictable file output, simplifying debugging and reducing ambiguity.
- **Layer Isolation** – Separating tile, collision, and event data prevents corruption in one subsystem from cascading into others.

Data Safety and Recovery

Controlled Persistence

- Autosave activates only after an initial manual save for new maps, ensuring user awareness of file creation.
- Save operations overwrite files in a controlled manner to avoid partial writes.
- Malformed files are detected during load, and execution is prevented until resolved.

Corruption Prevention

- Dimension consistency checks prevent mismatched grid structures.
- Validation is applied before serialisation to ensure only valid states are written to disk.
- Defensive error handling prevents malformed data from propagating into runtime systems.

Summary of Data Architecture

The data model is intentionally structured, versioned, and layer-isolated. It directly mirrors the conceptual structure of tile-based world design while maintaining runtime compatibility and performance alignment.

Through disciplined schema design and validation, the system ensures:

- Portability
- Structural clarity
- Rendering efficiency
- Data integrity
- Scalability

Without unnecessary architectural complexity.

Project Objectives

Objective 1: Core System Infrastructure and Foundational Architecture

1.1 Project Structure and Solution Architecture

1.1.1 Solution Configuration built within Visual Studio

- 1.1.1.1 Setup and finalise a “Master” solution to contain all editor, runtime, and shared model projects
- 1.1.1.2 Establish a Runtime class library to isolate core playback systems (rendering, input, collision, camera)

- 1.1.1.3 Create an Editor application project to serve as the primary map creation interface
- 1.1.1.4 Design and implement a Shared DTO project to enforce consistency across editor and runtime

1.1.2 Project Reference Configuration

- 1.1.2.1 Configure the Editor project to reference Shared DTOs for all map structures and serialisation contracts
- 1.1.2.2 Configure the Runtime project to reference Shared DTOs to ensure direct consumption of editor outputs
- 1.1.2.3 Implement a namespace hierarchy that clearly separates concerns (e.g. Vitriol.Editor, Vitriol.Engine, Vitriol.Shared)
- 1.1.2.4 Validate project dependency direction (Shared referenced by both, no circular coupling between Editor and Runtime)

1.1.3 Validation for the main Build System

- 1.1.3.1 Successfully execute clean builds of the entire solution in Release and Debug configurations
- 1.1.3.2 Ensure compilation output is structured correctly and does not rely on missing local dependencies
- 1.1.3.3 Create repeatable run configurations for both Editor and Runtime for testing and demonstration
- 1.1.3.4 Validate that the runtime can launch independently and load maps created by the editor

1.2 Management of Dependencies and Package Configuration

1.2.1 NuGet and Dependency Discipline

- 1.2.1.1 Integrate JSON serialisation tooling appropriate for DTO persistence (stable and maintainable)
- 1.2.1.2 Ensure WPF rendering dependencies remain minimal to preserve performance and deployment stability
- 1.2.1.3 Ensure release builds include all required DLLs and execute on clean systems

1.3 Object-Oriented Architecture and Separation of Concerns

1.3.1 Design of Core Class Hierarchy

- 1.3.1.1 Implement core map structures as DTOs (MapData, Layer data, Event definitions)
- 1.3.1.2 Implement clear runtime abstractions (Camera, Player, Input handling, Render systems)
- 1.3.1.3 Separate map rendering logic from map storage logic to prevent structural coupling
- 1.3.1.4 Ensure event objects and collision data remain independent of tile rendering data

1.3.2 Interface Segregation and Modular Components

- 1.3.2.1 Define interfaces for update loops and renderable systems to standardise runtime execution
- 1.3.2.2 Encapsulate editor tools as modular behaviours (tile brush, collision brush, event placement tool)
- 1.3.2.3 Ensure shared validation utilities are reusable between editor and runtime
- 1.3.2.4 Document component boundaries for maintainability and future extension

Objective 2: Map Data Model, Persistence, and Validation

2.1 Structured JSON Schema Design

2.1.1 Map Data Format Definition

- 2.1.1.1 Define a stable JSON structure containing metadata, tileset reference, and layered map data
- 2.1.1.2 Define tile layer format using tile indices referencing the loaded tileset
- 2.1.1.3 Define collision layer format as a grid aligned to the tile grid
- 2.1.1.4 Define event layer format as a list of position-based event objects with typed properties

2.1.2 Schema Versioning

- 2.1.2.1 Add a format version field to map files for controlled schema evolution
- 2.1.2.2 Ensure version field is validated during load to prevent undefined behaviour
- 2.1.2.3 Establish rules for forward-compatible extension (new fields appended without breaking existing)
- 2.1.2.4 Maintain clear documentation of schema meaning to support maintenance and testing

2.2 Persistence Pipeline

2.2.1 Save and Load System

- 2.2.1.1 Implement map serialisation to JSON from the shared DTO model
- 2.2.1.2 Implement deserialization with defensive error handling and clear failure states
- 2.2.1.3 Ensure tile and collision arrays match declared map dimensions before saving
- 2.2.1.4 Implement relative tileset path persistence to preserve portability between machines

2.2.2 Data Integrity Validation

- 2.2.2.1 Validate tile indices against tileset bounds during runtime/editor render preparation
- 2.2.2.2 Validate event positions fall within map boundaries
- 2.2.2.3 Validate collision grid dimensions match tile grid dimensions
- 2.2.2.4 Prevent invalid sates from being written to disk by validating prior to serialisation

2.3 Autosave Behaviour and Persistence Safety

2.3.1 Autosave Activation Rules

- 2.3.1.1 Implement rule: new maps require at least one manual save before autosave is enabled
- 2.3.1.2 Implement rule: existing saved maps may autosave on subsequent edits
- 2.3.1.3 Implement rule: sample/testing maps may autosave continuously for rapid iteration
- 2.3.1.4 Ensure autosave never overwrites an unintended file path or produces uncontrolled file duplication

2.3.2 Failure Handling

- 2.3.2.1 Handle failed saves gracefully (e.g. file locked, invalid path) without corrupting in-memory state
- 2.3.2.2 Prevent runtime load attempts of known-invalid map files
- 2.3.2.3 Ensure error messages indicate cause and resolution direction

- 2.3.2.4 Validate that recovery from invalid data does not require manual file editing by the user

Objective 3: Graphical Editor Interface System

3.1 Main Editor Window Architecture

3.1.1 WPF UI Structure

- 3.1.1.1 Implement a main viewport-based editor layout centred around map rendering
- 3.1.1.2 Implement UI controls for loading tilesets, creating maps, and switching editing modes
- 3.1.1.3 Ensure window resizing and DPI behaviour remains stable and readable
- 3.1.1.4 Provide clear visual distinction between tile editing, collision editing, and event editing modes

3.1.2 Workspace Interaction Model

- 3.1.2.1 Implement selection logic appropriate for tools (tile selection from palette, event selection on map)
- 3.1.2.2 Implement status feedback to show active tool, coordinates, and current layer/mode
- 3.1.2.3 Ensure tool interactions do not conflict (e.g. collision edits not painting tiles)
- 3.1.2.4 Validate workflow through structured user task sequences (create -> edit -> save -> load)

3.2 Editor Tooling and Usability Features

3.2.1 Tileset Handling and Palette

- 3.2.1.1 Load tileset images from disk and slice into tile-sized regions for selection
- 3.2.1.2 Implement a palette panel that displays tiles and support selection of current brush tile
- 3.2.1.3 Ensure selected tile index maps correctly to tileset coordinated during painting
- 3.2.1.4 Validate tileset replacement does not silently corrupt map tile indices

3.2.2 Mode-Specific Tools

- 3.2.2.1 Implement tile painting tool supporting click-to-paint and drag-to-paint behaviour
- 3.2.2.2 Implement collision painting tool that toggles blocked/walkable states
- 3.2.2.3 Implement even placement tool that creates and removes events at grid coordinates
- 3.2.2.4 Ensure each tool updates the viewport immediately to maintain user trust and feedback

Objective 4: Map Editor System

4.1 Tile-Based Rendering Engine (Editor)

4.1.1 Map Visualisation System

- 4.1.1.1 Implement a WPF rendering surface for tile map drawing
- 4.1.1.2 Implement coordinate conversion between mouse position and tile grid indices
- 4.1.1.3 Implement camera/viewport panning and zoom controls appropriate for large maps
- 4.1.1.4 Implement viewport-only rendering (render only visible tiles rather than full map)

4.1.2 Layer Rendering and Overlays

- 4.1.2.1 Render tile layer as the base visual layer
- 4.1.2.2 Render collision overlay with clear visual distinction from tiles
- 4.1.2.3 Render event makers with identifiable icons/visuals at their grid positions
- 4.1.2.4 Ensure overlays are toggleable to reduce visual clutter during specific editing tasks

4.2 Editing History System

4.2.1 Command Pattern Implementation

- 4.2.1.1 Define command objects for reversible edits (tile placement, collision toggle, event add/remove)
- 4.2.1.2 Track command execution order to support deterministic undo/redo
- 4.2.1.3 Implement Ctrl+Z / Ctrl+Y keyboard shortcuts for editing history navigation
- 4.2.1.4 Ensure undo/redo updates both data model and viewport rendering immediately

4.2.2 History Safety and Correctness

- 4.2.2.1 Ensure undo never produces invalid dimension states
- 4.2.2.2 Ensure redo matches original edit behaviour exactly (no divergence)
- 4.2.2.3 Ensure history operates consistently across all three map layers
- 4.2.2.4 Test edge cases (rapid drag edits, repeated undos, mixed layer editing sequences)

Objective 5: Runtime Playback System

5.1 Architecture of Core Runtime Loop

5.1.1 Engine Initialisation

- 5.1.1.1 Implement a runtime entry point that initialises rendering, input, and map loading subsystems
- 5.1.1.2 Load map JSON and tileset resources using the shared DTO model
- 5.1.1.3 Validate map integrity before entering gameplay loop
- 5.1.1.4 Ensure runtime starts reliably with a selected map and does not require editor context

5.1.2 Deterministic Update Loop

- 5.1.2.1 Implement a fixed timestep loop for consistent movement and collision evaluation
- 5.1.2.2 Separate update frequency from rendering frequency to ensure stability

- 5.1.2.3 Ensure consistent movement speed across different hardware
- 5.1.2.4 Validate long-runtime stability (no drift, no progressive slowdown)

5.2 Rendering and Camera System

5.2.1 Viewport Rendering

- 5.2.1.1 Render only visible tiles within the camera bounds to maintain performance
- 5.2.1.2 Enforce layer rendering order (map tiles -> events/objects -> player sprite)
- 5.2.1.3 Ensure correct tile-to-screen coordinate mapping at different zoom levels
- 5.2.1.4 Validate rendering correctness at map edges and during camera movement

5.2.2 Camera Follow Behaviour

- 5.2.2.1 Implement camera positioning to follow player movement while respecting map bounds
- 5.2.2.2 Ensure camera movement remains smooth and does not jitter during collision contact
- 5.2.2.3 Maintain consistent visible area calculations for viewport rendering
- 5.2.2.4 Validate camera behaviour on small maps where viewport exceeds map size

5.3 Player Movement, Collision, and Event Interaction

5.3.1 Input Handling

- 5.3.1.1 Implement keyboard input tracking for movement directions
- 5.3.1.2 Prevent inconsistent movement behaviour during rapid key presses
- 5.3.1.3 Ensure input is processed within fixed timestep updates for determinism
- 5.3.1.4 Validate movement speed remains consistent regardless of framerate

5.3.2 Collision Detection

- 5.3.2.1 Implement grid-based collision checks against the collision layer
- 5.3.2.2 Prevent entering blocked tiles and prevent clipping across tile boundaries
- 5.3.2.3 Ensure collision checks align exactly to editor-defined collision cells
- 5.3.2.4 Validate edge-case interactions at map borders and corners

5.3.3 Event Trigger Behaviour

- 5.3.3.1 Trigger events based on player position interacting with event coordinates
- 5.3.3.2 Ensure event activation conditions remain consistent and do not double-trigger unintentionally
- 5.3.3.3 Provide a clear runtime response to event activation (e.g. debug output/UI indicator)
- 5.3.3.4 Validate event persistence (events saved in editor load identically in runtime)

Objective 6: Performance, Stability, and Quality Assurance

6.1 Performance Validation Targets

6.1.1 Editor Performance

- 6.1.1.1 Maintain responsive painting behaviour on maps $\geq 100 \times 100$ tiles
- 6.1.1.2 Ensure viewport rendering prevents lag spikes during camera movement
- 6.1.1.3 Validate no progressive slowdown during extended editing sections
- 6.1.1.4 Validate tileset loading does not lock the UI unnecessarily

6.1.2 Runtime Performance

- 6.1.2.1 Maintain approximately 60 FPS on target specification hardware
- 6.1.2.2 Ensure camera and collision updates remain stable under continuous movement
- 6.1.2.3 Validate map load times remain within acceptable limits for typical map sizes
- 6.1.2.4 Ensure performance does not degrade due to avoidable allocations or resource leaks

6.2 Reliability and Data Integrity

6.2.1 File Integrity

- 6.2.1.1 Ensure save/load round-trip does not alter map structure unexpectedly
- 6.2.1.2 Ensure invalid files fail safely and do not crash editor or runtime
- 6.2.1.3 Validate autosave does not corrupt map files during frequent edits
- 6.2.1.4 Ensure relative tileset paths preserve portability across devices

6.2.2 Stability

- 6.2.2.1 Validate that editor and runtime remain stable during repeated open/edit/save cycles
- 6.2.2.2 Validate that undo/redo cannot produce corrupted in-memory state
- 6.2.2.3 Validate that error handling paths are exercised and predictable
- 6.2.2.4 Maintain clear logs/messages for debugging during development and testing

Objective 7: Testing and Verification

7.1 Functional Test Planning

7.1.1 Core Workflow Test Cases

- 7.1.1.1 Create tests for map creation, tile painting, collision editing, event placement, and saving
- 7.1.1.2 Create tests for map loading, tileset loading, and schema validation
- 7.1.1.3 Create tests for undo/redo correctness across mixed edit sequences
- 7.1.1.4 Create tests for autosave rule enforcement (new map vs saved map vs sample map)

7.1.2 Runtime Verification

- 7.1.2.1 Create tests for runtime map load correctness matching editor output
- 7.1.2.2 Create tests for collision behaviour accuracy against collision layer definitions
- 7.1.2.3 Create tests for camera follow behaviour and edge-bound constraints
- 7.1.2.4 Create tests for event trigger activation conditions and persistence fidelity

7.2 Performance and Stability Testing

7.2.1 Performance Benchmarks

- 7.2.1.1 Measure runtime FPS stability on target specification hardware profile
- 7.2.1.2 Measure map load time for 100x100 and larger maps
- 7.2.1.3 Measure editor responsiveness during drag painting across large regions
- 7.2.1.4 Record memory behaviour across extended sessions to detect leaks

7.2.2 Failure and Recovery Testing

- 7.2.2.1 Test corrupted JSON input handling and ensure graceful failure
- 7.2.2.2 Test missing tileset reference handling
- 7.2.2.3 Test rapid undo/redo stress conditions
- 7.2.2.4 Test autosave interruption scenarios (e.g. sudden close) and ensure integrity remains preserved

Objective 8: Documentation and Evaluation Evidence

8.1 Technical Documentation

8.1.1 Documentation of Code

- 8.1.1.1 Implement structured inline documentation for public classes and key systems
- 8.1.1.2 Produce a clear explanation of architecture boundaries (Editor vs Runtime vs Shared)
- 8.1.1.3 Document design rationale for viewport-only rendering, layer separation, and command-based history
- 8.1.1.4 Maintain records of key implementation decisions and trade-offs to support evaluation

8.1.2 Documentation of the System

- 8.1.2.1 Create UML or component diagrams describing subsystem interactions
- 8.1.2.2 Document the JSON schema structure including layer definitions and version field
- 8.1.2.3 Provide diagrams o explanations of runtime loop and camera/viewport calculation
- 8.1.2.4 Document collision and event data representations with examples

8.2 Evaluation Evidence Compilation

8.2.1 Evidence Collection and Organisation

- 8.2.1.1 Capture annotated screenshots of editor features (tile, collision, event modes) as implementation evidence
- 8.2.1.2 Capture annotated screenshots of runtime playback to demonstrate editor-runtime cohesion
- 8.2.1.3 Store representative map JSON examples showing schema structure, versioning, and layer separation
- 8.2.1.4 Maintain a structured folder of test outputs, benchmark results, and failure/retest logs

8.2.2 Linking Evidence to Criteria

- 8.2.2.1 Map test cases directly to functional requirements IDs (FR1-FR12)
- 8.2.2.2 Map benchmarks (FPS/load times) directly to non-functional requirements targets
- 8.2.2.3 Record notable bug fixes as “issue -> fix -> retest” entries for evaluation credibility

- 8.2.2.4 Ensure each success criterion claim is backed by at least one concrete piece of evidence

8.3 Final Self-Evaluation Framework

- 8.3.1.1 Create an evaluation comparing success criteria against implemented outcomes
- 8.3.1.2 Record measurable benchmarks (load times, FPS stability, responsiveness on large maps)
- 8.3.1.3 Present scope boundaries as deliberate design constraints (clarity and performance focus)
- 8.3.1.4 Present testing evidence using structure test cases and retest outcomes

Success Criteria

Functional Completeness Metrics

Core Feature Implementation

- **Map Editing Completion** – Creation, editing, saving, and loading of maps must function with no structural data corruption. Tile, collision, and event layers must persist accurately across save/load cycles.
- **Editor-Runtime Cohesion** – Maps created in the editor must load into the runtime without requiring modification or manual conversion. Runtime behaviour (collision, event triggers, rendering order) must reflect editor configuration exactly.
- **Undo/Redo Reliability** – All reversible editing actions (tile paint, collision toggle, event placement/removal) must be fully reversible with deterministic behaviour and no divergence after repeated undo/redo sequences.
- **Autosave Rule Compliance** – Autosave must activate only under defined conditions (post-manual-save for new maps), and must never create unintended file duplication or overwrite unintended files.

Performance Benchmark Compliance

Runtime Performance Standards

- **Frame Rate Stability** – Runtime must maintain approximately 60 FPS ($\pm 5\%$ variance) on minimum target hardware (Intel UHD 620, 8GB RAM) during standard gameplay movement on maps $\geq 100 \times 100$ tiles.
- **Loading Time Targets** – Maps of size 100×100 must load within 2 seconds under standard test conditions.
- **Rendering Scalability** – Viewport-only rendering must prevent full-map redraw overhead, ensuring stable performance regardless of total map size beyond visible region.
- **Editor Responsiveness** – Drag-to-paint operations must remain visually responsive with no perceptible input lag on large maps.

User Experience Success Indicators

Learnability and Accessibility

- **First-Time Workflow Completion** – A user unfamiliar with the system must be able to:
Create a map -> Paint tiles -> Add collision -> Place an event -> Save -> Load in runtime
within a single session without external intervention.
- **Task Completion Rate** – At least 85% successful completion rate for core workflow tasks during structured testing sessions.
- **Error Recovery Success** – Users must be able to recover from common issues (e.g., invalid tileset path, accidental edits) using undo or error prompts without external debugging.

Workflow Efficiency Indicators

- **Reduced Cognitive Switching** – Editing of tiles, collision, and events must occur within a single unified viewport without switching to external tools.
- **SUS Target** – System Usability Scale score target $\geq 80\%$ across testing participants.
- **Confidence Growth Indicator** – Users should report increased confidence after completing a full create-edit-play cycle.

Technical Quality Measures

Code Quality and Maintainability

- **Architecture Separation Compliance** – Clear separation must exist between:
 - Editor logic
 - Runtime logic
 - Shared data modelsNo circular dependencies between major components
- **Structured Namespace Organisation** – Namespaces must reflect architectural boundaries and maintain clarity of responsibility.
- **Consistent Coding Standards** – No critical compilation warnings in Release build configuration.
- **Documented Design Rationale** – Key architectural decisions (viewport rendering, layered data model, command pattern) must be documented for evaluation.

Reliability and Stability

Runtime Stability

- **Crash Frequency** – Fewer than 2 crashes per 100 hours of structured testing.

- **Deterministic Behaviour** – Movement speed, collision resolution, and event triggering must behave consistently across repeated runs on identical maps.

Data Integrity

- **Zero Data Corruption in Normal Usage** – Save -> Close -> Reload cycle must not alter tile, collision, or event data.
- **Validation Coverage** – Invalid JSON structures must fail gracefully without crashing the application.
- **Autosave Safety** – Autosave must not produce corrupted partial writes during frequent edits.

Long-Term Viability Indicators

Scalability

- **Map Size Handling** – Maps $\geq 100 \times 100$ tiles must remain performant in both editor and runtime environments.
- **Rendering Efficiency** – Performance must scale based on visible viewport rather than total map size.
- **Extensibility Foundation** – The presence of schema versioning and modular architecture must allow new layer types or map properties to be added without restructuring core systems.

Documentation and Evidence

- **Technical Documentation Completion** – Core architectural systems, JSON schema, runtime loop, and rendering logic must be clearly documented.
- **Testing Evidence Availability** – All major functional and performance claims must be supported by structured test cases and recorded results.
- **Evaluation Traceability** – Each functional and non-functional requirement must map to at least one objective and one test case.

Traceability Matrix

Project Focus Area	Supporting Objectives	Linked Functional Requirements
System Architecture	Obj 1.1, 1.3	FR5, FR8, FR11, FR12
Map Data & Persistence	Obj 2.1, 2.2, 2.3	FR1, FR5, FR9, FR11
Editor Interface	Obj 3.1, 3.2	FR1, FR2, FR3, FR4, FR8
Map Rendering Engine (Editor)	Obj 4.1	FR2, FR6, FR12

Editing History System	Obj 4.2	FR8
Runtime Engine	Obj 5.1, 5.2	FR6, FR7, FR10, FR12
Collision & Events	Obj 5.3	FR3, FR4, FR7
Performance Validation	Obj 6.1	FR10
Reliability & Integrity	Obj 6.2	FR5, FR8, FR9
Testing & Verification	Obj 7.1, 7.2	All FRs
Documentation & Evidence	Obj 8.1, 8.2	All FRs